Innopolis University

Big Data Final Project Report

Team Members:

a.nasibullina@innopolis.university

v.andruwenko@innopolis.university

u.chaikouskaya@innopolis.university

n.shiyanov@innopolis.university

Big Data Analytics on NYC Yellow Taxi Trip Records

May 12, 2026

Contents

1 Introduction **3**
- 1.1 Business Objectives . . . 3
- 1.2 Project Scope . . . 3

2 Data Description **3**
- 2.1 Data Characteristics . . . 4
- 2.2 Feature Names and Descriptions . . . 4

3 Architecture of Data Pipeline **5**
- 3.1 Pipeline Overview . . . 5
- 3.2 Pipeline Inputs and Outputs . . . 5

4 Stage 1: Data Preparation **6**
- 4.1 Relational Model and ER Diagram . . . 6
- 4.2 Database Samples . . . 7
- 4.3 HDFS Location and Storage Format (Sqoop) . . . 8

5 Stage 2: Data Analysis **8**
- 5.1 Preparing the Hive Environment . . . 8
- 5.2 Creating the External Base Table . . . 8
- 5.3 Creating the Partitioned and Bucketed Table . . . 9
- 5.4 EDA Queries . . . 10
- 5.5 Exporting Results and Visualising . . . 11

6 Results **11**
- 6.1 Dataset Overview and Data Quality (Q1, Q2) . . . 11
- 6.2 Passenger Count and Fare Distribution (Q3) . . . 12
- 6.3 Trip Distance Distribution (Q4) . . . 12
- 6.4 Revenue Components (Q5) . . . 13
- 6.5 Payment Methods (Q6) . . . 14
- 6.6 Hourly Demand (Q7) . . . 15
- 6.7 Trip Types and Airport Premiums (Q8) . . . 16
- 6.8 Revenue per Mile (Q9) . . . 16
- 6.9 Weekly Patterns (Q10) . . . 17
- 6.10 Top Pickup Locations (Q11) . . . 17

7 Discussion **17**

8 Conclusion **17**

9 Stage 3: ML Modeling **17**
- 9.1 Feature Extraction and Data Preprocessing . . . 18
- 9.2 Training and Fine-Tuning . . . 18
- 9.3 Evaluation . . . 19
- 9.4 Prediction for a Specific Sample . . . 20

10 Stage 4: Data Presentation **21**
10.1 Dashboard Overview 21
10.2 Chart Descriptions 22
10.3 Findings 24

11 Conclusion **24**

12 Testing and Quality Assurance **25**
12.1 Static Tests (GitHub Actions CI) 25
12.2 Cluster Verification Scripts 26
12.3 Testing Challenges 26

13 Reflections on Own Work **26**
13.1 Challenges and Difficulties 26
13.2 Recommendations 27
13.3 Team Contributions 27

1 Introduction

This project analyzes NYC Yellow Taxi trip records for 2025. The goal is to predict the tip amount a passenger will leave for the driver, which makes the predictive analysis a regression task. Why do we need this? Estimating the expected tip from known or estimated trip characteristics helps evaluate how valuable different ride patterns are. A higher predicted tip means a more profitable ride, which allows drivers and analysts to make smarter decisions about locations, time periods, and trip types.

1.1 Business Objectives

The primary business objective of this project is to support taxi drivers and transportation analysts in understanding which trip characteristics are associated with higher tips. Since tips directly affect a driver's final income, reliable tip prediction can help identify trips, locations, and time periods with higher expected profitability and improve revenue planning.

From a technical perspective, the objective is to build a reproducible big data pipeline that stores, processes, analyzes, and models large-scale taxi trip records using distributed systems. The final dashboard should combine descriptive analytics and machine learning results so that business stakeholders can inspect both historical patterns and predictive performance.

1.2 Project Scope

The scope of the project covers NYC Yellow Taxi trip records for 2025. The input data contains pickup and drop-off timestamps, location identifiers, trip distances, fares, payment information, passenger counts, and related trip attributes. The target variable for predictive modeling is the tip amount, so the machine learning task is treated as a regression problem.

2 Data Description

Yellow taxi trip records include fields capturing pickup and drop-off dates/times, pickup and drop-off locations, trip distances, itemized fares, rate types, payment types, and driver-reported passenger counts.

2.1 Data Characteristics

Characteristic	Description
Dataset name	TLC Trip Record Data
Source	https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
Time period	2025 Year
Number of records	48,722,573
Dataset size	791.5 MB (0.77 GB)
Number of features	20 raw table columns, extended with engineered ML features
Prediction target	Tip amount.
Task type	Regression.
Datetime features	tpep_pickup_datetime, tpep_dropoff_datetime
Geospatial features	PULocationID, DOLocationID.

Table 1: Dataset characteristics

2.2 Feature Names and Descriptions

Feature	Type	Description
trip_id	BIGINT / identifier	Unique trip identifier generated in PostgreSQL. It is used to track records across ingestion, Hive storage, and validation outputs.
vendorid	INTEGER	Identifier of the taxi technology provider. It is useful for checking whether trip patterns or data quality differ by provider.
tpep_pickup_datetime	TIMESTAMP / BIGINT in Hive	Pickup timestamp. It supports time-based analysis such as hourly demand, day-of-week trends, monthly patterns, and derived ML features.
tpep_dropoff_datetime	TIMESTAMP / BIGINT in Hive	Drop-off timestamp. Together with pickup time, it is used to calculate trip duration for data quality filtering.
passenger_count	DOUBLE	Number of passengers reported by the driver. It is used to analyze group-size patterns and as a predictive feature for tipping behavior.
trip_distance	DOUBLE	Trip distance in miles. It is one of the main explanatory variables for fare, duration, revenue per mile, and expected tip behavior.
ratecodeid	DOUBLE	Final rate code for the trip, such as standard fare or airport-related fare. It helps distinguish trip types with different pricing and tipping patterns.
store_and_fwd_flag	STRING	Indicates whether trip data was temporarily stored before being sent to the vendor. It is mainly a data collection and quality indicator.
pulocationid	INTEGER	Taxi zone where the trip started. It supports pickup hotspot analysis and location-based demand or tipping comparisons.
dolocationid	INTEGER	Taxi zone where the trip ended. It supports destination analysis and origin-destination pattern interpretation.
payment_type	DOUBLE	Payment method code. It is important because credit card trips expose recorded tips and are filtered in Stage 3 for tip-based modeling.
fare_amount	DOUBLE	Base fare calculated by distance and time. It is a key revenue component and one of the strongest predictors of the absolute tip amount.
extra	DOUBLE	Additional charges such as rush-hour or overnight surcharges. It helps explain total trip cost and passenger payment context.
mta_tax	DOUBLE	MTA tax charged on eligible trips. It is part of the fare breakdown used in revenue component analysis.
tip_amount	DOUBLE	Tip paid by the passenger. It is the central target field for Stage 3 regression and is also used in EDA tipping analyses.
tolls_amount	DOUBLE	Toll charges paid during the trip. It helps explain high-cost rides and differences in total amount.
improvement_surcharge	DOUBLE	Fixed surcharge applied to eligible taxi trips. It is included in the total payment breakdown.
total_amount	DOUBLE	Total amount charged to the passenger, including fare, taxes, surcharges, tolls, and tips. It is used for revenue-level analysis.
congestion_surcharge	DOUBLE	Congestion surcharge for trips in applicable zones. It captures pricing impact for congested areas such as Manhattan.
airport_fee	DOUBLE	Airport fee applied to airport trips. It helps identify and explain airport-related fare patterns.

Feature	Type	Description
month	INTEGER, derived	Hive partition column extracted from pickup time (month 1–12). Monthly partitioning improves query pruning on a single-year dataset compared to yearly partitioning.
trip_duration_min	DOUBLE, derived	Trip duration in minutes calculated from pickup and drop-off timestamps. It is used in Stage 3 to remove trips with unrealistic durations before modeling.
pickup_month_sin, pickup_month_cos	DOUBLE, derived	Cyclical month encoding used by Stage 3 so the model can represent seasonality without treating December and January as far apart.
pickup_hour_sin, pickup_hour_cos	DOUBLE, derived	Cyclical hour encoding used to capture daily tipping and demand patterns.
pickup_dow_sin, pickup_dow_cos	DOUBLE, derived	Cyclical day-of-week encoding used to represent weekly patterns in passenger behavior.
is_weekend	DOUBLE, derived	Binary indicator showing whether pickup happened on Saturday or Sunday. It helps compare weekend and weekday trip behavior.
rush_hour	DOUBLE, derived	Binary indicator for morning and evening rush-hour pickups. It captures commuter-period travel behavior.
night_trip	DOUBLE, derived	Binary indicator for late-night trips. It supports analysis of whether night rides have different tipping patterns.
airport_rate	DOUBLE, derived	Binary indicator for JFK or Newark rate codes. It helps separate airport trips from standard urban trips.
pre_tip_amount	DOUBLE, derived	Sum of fare, taxes, surcharges, tolls, and fees before tip. It describes the passenger's charge context before gratuity.
label	DOUBLE, derived target	Stage 3 model label created by renaming tip_amount; it is the regression target predicted by Spark ML models.

Table 2: Dataset features

3 Architecture of Data Pipeline

3.1 Pipeline Overview

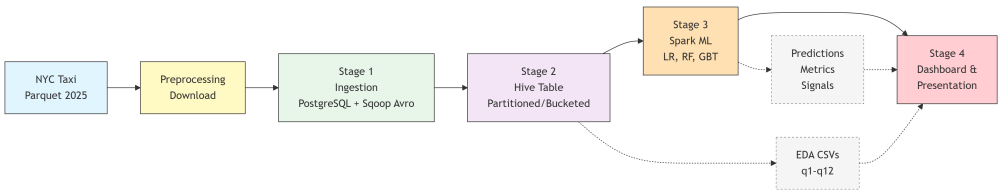

Figure 1: End-to-end data pipeline architecture

3.2 Pipeline Inputs and Outputs

Stage	Input	Output	Location
Preprocessing: data download	NYC TLC Yellow Taxi monthly Parquet files for 2025	Local Parquet dataset used by Stage 1	data/yellow_tripdata_2025-*.parquet
Stage 1: data collection and ingestion	Local Yellow Taxi Parquet files	PostgreSQL table yellow_taxi_trips; Sqoop-imported Avro files compressed with Snappy; generated Avro schema and Java class	PostgreSQL database $USER_projectdb HDFS project/warehouse/yellow_taxi_trips local output/yellow_taxi_trips.avsc local output/sqoop_codegen/
Stage 2: data storage and EDA preparation	Sqoop Avro files and output/*.avsc	Hive database team35_projectdb; external partitioned and bucketed table yellow_taxi_trips_part_buck; EDA result tables and exported CSV files	HDFS project/hive/warehouse/yellow_taxi_trips_part_buck HDFS project/warehouse/avsc local output/q1.csv–output/q11.csv local output/hive_results.txt local output/eda_results.txt

Stage	Input	Output	Location
Stage 3: predictive data analysis	Hive table team35_projectdb	Trained Spark ML regression models; prediction CSVs; evaluation metrics; feature signal export; training summary	models/model1_lr models/model2_rf models/model3_gbt output/model1_predictions.csv output/model2_predictions.csv output/model3_predictions.csv output/evaluation.csv output/model_feature_signals.csv output/stage3_training_summary.json
Stage 4: data presentation	EDA CSV files, Stage 3 metrics, model predictions, and feature signals	Dashboard and presentation materials for communicating dataset characteristics, EDA insights, and model performance	–

Table 3: Pipeline inputs and outputs

4 Stage 1: Data Preparation

In Stage 1, NYC Yellow Taxi trip records for 2025 were collected under `./data/`, loaded into PostgreSQL on the IU Hadoop cluster, exported to HDFS with Sqoop as Snappy-compressed Avro files, and thereby prepared as the raw distributed storage layer for subsequent Hive and Spark analytics.

The monthly Parquet files (`yellow_tripdata_2025-*.parquet`) were downloaded into `./data/` (combined size $\approx$ 792 MB) using `bash scripts/preprocess.sh` on `hadoop-01`. Loading was performed with `python3 scripts/build_projectdb.py`: Parquet is read in batches; TLC column names are mapped to the lowercase SQL schema; timestamps are normalized to `timestamp without time zone`; numeric fields are coerced to the declared types; string fields such as `store_and_fwd_flag` are truncated where needed; and rows are bulk-inserted with PostgreSQL `COPY`. Sqoop then imports the `yellow_taxi_trips` table into HDFS as Avro tables compressed with Snappy.

4.1 Relational Model and ER Diagram

The relational schema consists of a single fact table `yellow_taxi_trips` in the `public` schema of the database `team35_projectdb` on host `hadoop-04.uni.innopolis.ru`.

Key characteristics:

- **Primary key:** `trip_id` (`BIGSERIAL`), surrogate key assigned on insert.
- **Columns:** attributes follow TLC semantics; names and types match `sql/create_tables.sql` and Figure 2. Grouped for readability:
  - trip context: `vendorid`; `tpep_pickup_datetime`, `tpep_dropoff_datetime`; `passenger_count`, `trip_distance`, `ratecodeid`, `store_and_fwd_flag`;
  - TLC zones: `pulocationid`, `dolocationid`; `payment_type`;
  - fares and charges: `fare_amount`, `extra`, `mta_tax`, `tip_amount`, `tolls_amount`, `improvement_surcharge`, `total_amount`, `congestion_surcharge`, `airport_fee`.
- **Constraints:** primary key on `trip_id`; remaining columns use the types in `sql/create_tables.sql` (`INTEGER`, `DOUBLE PRECISION`,

`TIMESTAMP WITHOUT TIME ZONE`, `VARCHAR(8)`). All columns except the surrogate key are nullable as in the DDL.

- **Deployment:** single PostgreSQL instance on the cluster (no Citus sharding).

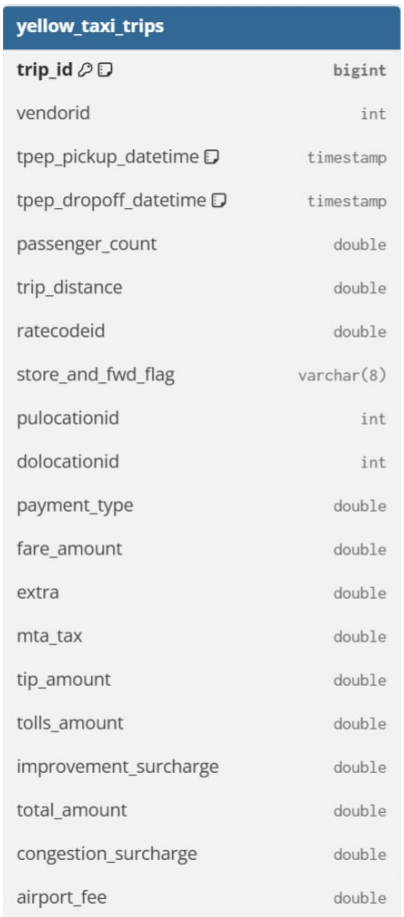

Figure 2: Entity-relationship diagram for the `yellow_taxi_trips` table

4.2 Database Samples

Table	Sample description
`yellow_taxi_trips`	Fact table containing 48 722 602 Yellow Taxi trip rows for 2025 after the full load.

Table 4: Database table samples

A representative preview of the data is shown in Figure 3 (obtained with `SELECT * FROM yellow_taxi_trips ORDER BY trip_id LIMIT 5;`).

trip_id [PK] bigint	vendorid integer	tpep_pickup_datetime timestamp without tim	tpep_dropoff_datetime timestamp without time	passenger_co double precis	trip_distance double precis	pulocationid integer	dolocationid integer	payment_type double precisi	fare_amount double precis	tip_amount double precis	total_amount double precisi
1	1	2025-01-01 00:18:38	2025-01-01 00:26:59	1	1.6	229	237	1	10	3	18
2	1	2025-01-01 00:32:40	2025-01-01 00:35:13	1	0.5	236	237	1	5.1	2.02	12.12
3	1	2025-01-01 00:44:04	2025-01-01 00:46:01	1	0.6	141	141	1	5.1	2	12.1

Figure 3: Example rows from the `yellow_taxi_trips` table

These samples illustrate typical short trips paid by credit card. They demonstrate the main attributes used for exploratory analysis and tip amount prediction: timestamps, trip distance, location IDs, fare components, and the target variable `tip_amount`.
A representative preview is obtained with `SELECT * FROM yellow_taxi_trips ORDER BY trip_id LIMIT 5;`, showing timestamps, zone IDs, trip distance, fare components, and `tip_amount`, among other attributes.

4.3 HDFS Location and Storage Format (Sqoop)

The PostgreSQL table `yellow_taxi_trips` was imported with Sqoop into HDFS at:
`/user/team35/project/warehouse/yellow_taxi_trips`
The directory contains nine objects: a zero-byte `_SUCCESS` marker and eight Avro part files `part-m-00000.avro`–`part-m-00007.avro`. The sum of part-file sizes in this directory is about 2.3 GB of application data (before HDFS replication); with the default replication factor of three, total block usage reported by HDFS is roughly three times larger.

Artifacts for downstream stages include the Avro schema `output/yellow_taxi_trips.avsc` and Sqoop-generated Java sources under `output/sqoop_codegen/`.

Format choice: Avro embeds the schema with the data, which fits Hadoop and Sqoop workflows; Snappy trades maximum compression ratio for fast compression and decompression on large scans.

5 Stage 2: Data Analysis

5.1 Preparing the Hive Environment

Before any Hive tables could be created, the Avro schema file had to be placed in HDFS so that the Avro SerDe can locate it at query time. The schema file generated by Sqoop in Stage 1 was uploaded to a dedicated HDFS directory:

```
hdfs dfs -mkdir -p project/warehouse/avsc
hdfs dfs -put -f output/yellow_taxi_trips.avsc \
                project/warehouse/avsc/
```

The Hive database `team35_projectdb` was then created in a location separate from the Sqoop import path to avoid any interference with the data imported in Stage 1. The database stores its metadata in the Hive metastore while its table data lives under `project/hive/warehouse` in HDFS:

```
DROP DATABASE IF EXISTS team35_projectdb CASCADE;
CREATE DATABASE team35_projectdb
    LOCATION 'project/hive/warehouse';
USE team35_projectdb;
```

5.2 Creating the External Base Table

An external Hive table was mapped directly onto the Avro files produced by Sqoop. Using the EXTERNAL keyword is important because it means that dropping the

table from Hive's metastore will not delete the underlying HDFS files — the data remains intact and can be re-registered at any time:

```
CREATE EXTERNAL TABLE yellow_taxi_trips
STORED AS AVRO
LOCATION 'project/warehouse/yellow_taxi_trips'
TBLPROPERTIES (
    'avro.schema.url'='project/warehouse/avsc/yellow_taxi_trips.avsc'
);
```

This table was used only as a staging source for the next step and was dropped after the data was transferred.

5.3 Creating the Partitioned and Bucketed Table

The main analytical table used for all EDA queries is a partitioned and bucketed variant of the base table. Partitioning was done by `month` (values 1 to 12, derived from the pickup timestamp), which produces 12 distinct partitions — one for each calendar month of 2025. This design was chosen deliberately: partitioning by `year` would produce only a single partition for a single-year dataset, providing no pruning benefit whatsoever. With monthly partitions, any query that filters on a specific month or month range causes Hive to skip the irrelevant partitions entirely, reducing I/O by up to 11/12. Within each partition, the data is further organised into 4 buckets by the vendorid column, which enables efficient bucket-map joins in later stages.

Dynamic partitioning in nonstrict mode was enabled so that a single `INSERT` statement could populate all 12 partitions at once, without needing to write a separate insert for each month:

Listing 1: Partitioned and bucketed table creation (`sql/db.hql`).

```
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;

DROP TABLE IF EXISTS yellow_taxi_trips_part_buck;

CREATE EXTERNAL TABLE yellow_taxi_trips_part_buck (
    trip_id                 BIGINT,
    vendorid                INT,
    tpep_pickup_datetime    BIGINT,
    tpep_dropoff_datetime   BIGINT,
    passenger_count         DOUBLE,
    trip_distance           DOUBLE,
    ratecodeid              DOUBLE,
    store_and_fwd_flag      STRING,
    pulocationid            INT,
    dolocationid            INT,
    payment_type            DOUBLE,
    fare_amount             DOUBLE,
    extra                   DOUBLE,
    mta_tax                 DOUBLE,
```

```
    tip_amount                  DOUBLE,
    tolls_amount                DOUBLE,
    improvement_surcharge       DOUBLE,
    total_amount                DOUBLE,
    congestion_surcharge        DOUBLE,
    airport_fee                 DOUBLE
)
PARTITIONED BY (month INT)
CLUSTERED BY (vendorid) INTO 4 BUCKETS
STORED AS AVRO
LOCATION 'project/hive/warehouse/yellow_taxi_trips_part_buck'
TBLPROPERTIES (
    'avro.schema.url'='project/warehouse/avsc/yellow_taxi_trips.avsc'
);
```

5.4 EDA Queries

All eleven EDA queries follow the same structural pattern: an existing result table is dropped and recreated as a CSV-formatted external table, the analytical query is executed and its results are inserted, and finally a `SELECT *` verifies the output. Every query uses `WHERE month BETWEEN 1 AND 12` as the partition predicate to address all twelve monthly partitions of the 2025 dataset. The queries are stored in `sql/eda.hql` and are described briefly below alongside their HiveQL code.

Q1 computes a dataset-level overview: total trip count, earliest and latest pickup timestamps, total revenue, average fare, average distance, and average tip.

Q2 counts the number of records affected by each of six quality issues: NULL datetimes, zero trip distance, zero or negative fare, zero passenger count, distance outliers above 50 miles, and fare outliers above $500.

Q3 groups trips by passenger count and computes average fare and tip for each group, ordered by descending trip volume.

Q4 assigns each trip to a labelled distance bucket (zero, under 1 mile, 1–3 miles, 3–7 miles, 7–15 miles, 15–50 miles, and over 50 miles) using a `CASE` expression, then counts trips and computes averages within each bucket.

Q5 aggregates the total amount for each revenue component in a single pass over the table: base fare, tip, MTA tax, tolls, improvement surcharge, congestion surcharge, airport fee, extra charges, and the grand total.

Q6 groups trips by payment type and computes trip count, total revenue, average fare, and average tip for each type.

Q7 extracts the hour of day from the pickup timestamp using `SUBSTRING` on the formatted string (since calling `HOUR()` on a `BIGINT` column is not supported), then aggregates trip count, average revenue, average tip, and total revenue for each of the 24 hours.

Q8 decodes the `ratecodeid` column into human-readable trip-type labels (Standard, JFK Airport, Newark Airport, Nassau/Westchester, Negotiated, Group ride) and compares average distance, fare, total charge, and tip across these categories.

Q9 computes revenue per mile for each distance bucket by dividing the average fare by the average trip distance, using `NULLIF` to guard against division by zero. This metric reveals how pricing efficiency changes with trip length.

Q10 determines the day of the week for each trip without relying on `DATE_FORMAT`, which has known pattern-compatibility issues across Hive/Java builds. Instead, it uses `DATEDIFF` against a known Sunday anchor date followed by a modulo-7 operation to derive the weekday index.

Q11 ranks TLC pickup zones by total trip count and returns the top 10, along with average fare and average tip for each zone.

5.5 Exporting Results and Visualising

After all queries ran successfully, each result table was exported to a local CSV file using Beeline's output redirection. The `stage2.sh` script defines a reusable `export_table` function that handles this uniformly for all eleven queries:

Charts were then produced manually in Apache Superset. For each query, a Hive-backed dataset was registered, an appropriate chart type was configured, and the chart was exported as a JPEG image stored in the `output/` folder.

6 Results

6.1 Dataset Overview and Data Quality (Q1, Q2)

The dataset spans the full year 2025 without any gaps in pickup timestamps. Across 48,722,573 trips, passengers collectively paid approximately $1.31 billion, with a mean base fare of $18.42 and a mean tip of $2.84. The average trip covered 6.84 miles, suggesting a mix of short city hops and longer inter-borough or airport journeys.

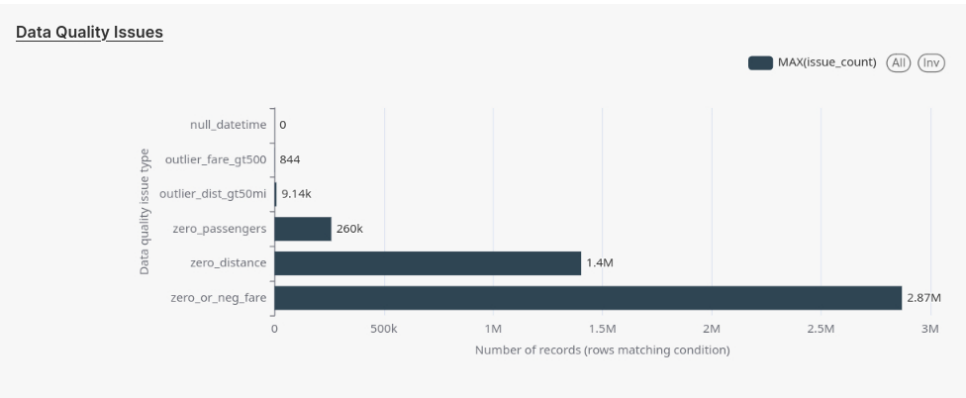

Figure 4: Data quality issue counts across key dimensions of the dataset (Q2).

The Q2 quality scan found no NULL pickup datetimes, which means all time-based analyses are reliable. The main issues are zero-or-negative fares (2.87 million records, 5.9% of the dataset) and zero trip distances (1.4 million records, 2.9%). These records most likely correspond to cancelled rides, voided meter activations, or data entry artefacts from the vendor systems and should be excluded from any fare or distance modelling. A separate concern is the 11.6 million records with a NULL passenger count, which appear to originate from a single vendor whose upstream data pipeline was not recording this field.

6.2 Passenger Count and Fare Distribution (Q3)

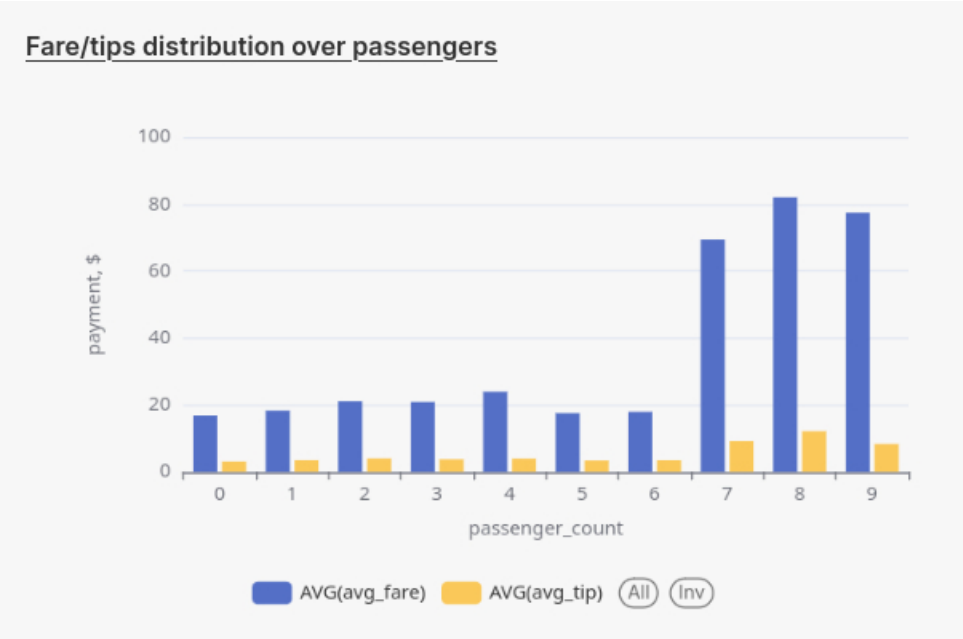

Figure 5: Average fare and tip by passenger count (Q3)

Solo passengers account for 29.4 million rides — 60% of all trips with a valid passenger count — and pay an average fare of $18.36. Groups of four passengers generate the highest average fare at $24.00, hinting that larger groups tend to travel longer distances, possibly to airports or outer boroughs. Notably, four-passenger groups also tip generously at $3.98 on average, only slightly below the two-passenger peak of $4.03.

6.3 Trip Distance Distribution (Q4)

The majority of NYC yellow taxi trips are short: 45.8% fall in the 1–3 mile range (average distance 1.78 miles, average fare $12.99), and an additional 20.1% are under one mile. Only about 4% of trips exceed 15 miles, and those trips command a substantially higher average fare of $68.57. The "over 50 miles" bucket contains just 9,159 records but shows an impossible average distance of 18,344 miles — a clear sign that these are data errors rather than real journeys and should be filtered out in any downstream modelling.

6.4 Revenue Components (Q5)

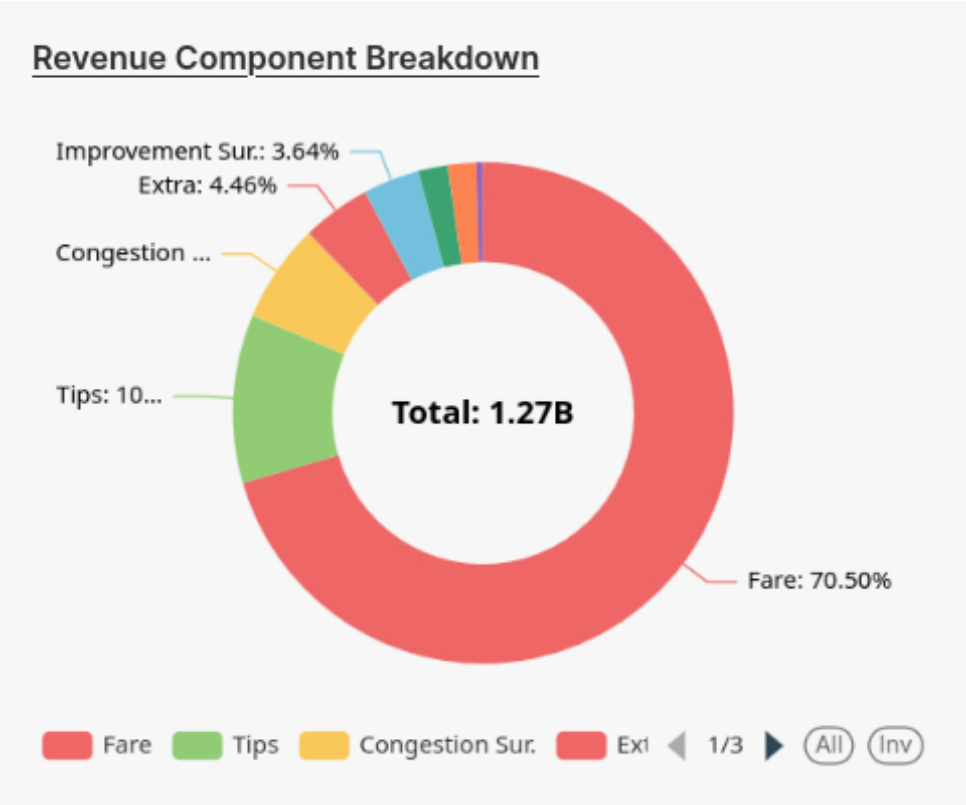

Figure 6: Breakdown of total 2025 revenue by component (Q5).

The base fare dominates the revenue picture, contributing \$897 million (68.5% of the \$1.31 billion grand total). Tips are the second-largest component at \$138 million (10.6%), followed by congestion surcharges (\$81 million), improvement surcharges (\$46 million), extra charges (\$57 million), and tolls (\$24 million). The airport fee contributes a modest \$5.4 million, reflecting the subset of trips that specifically originate from or terminate at LaGuardia or JFK.

6.5 Payment Methods (Q6)

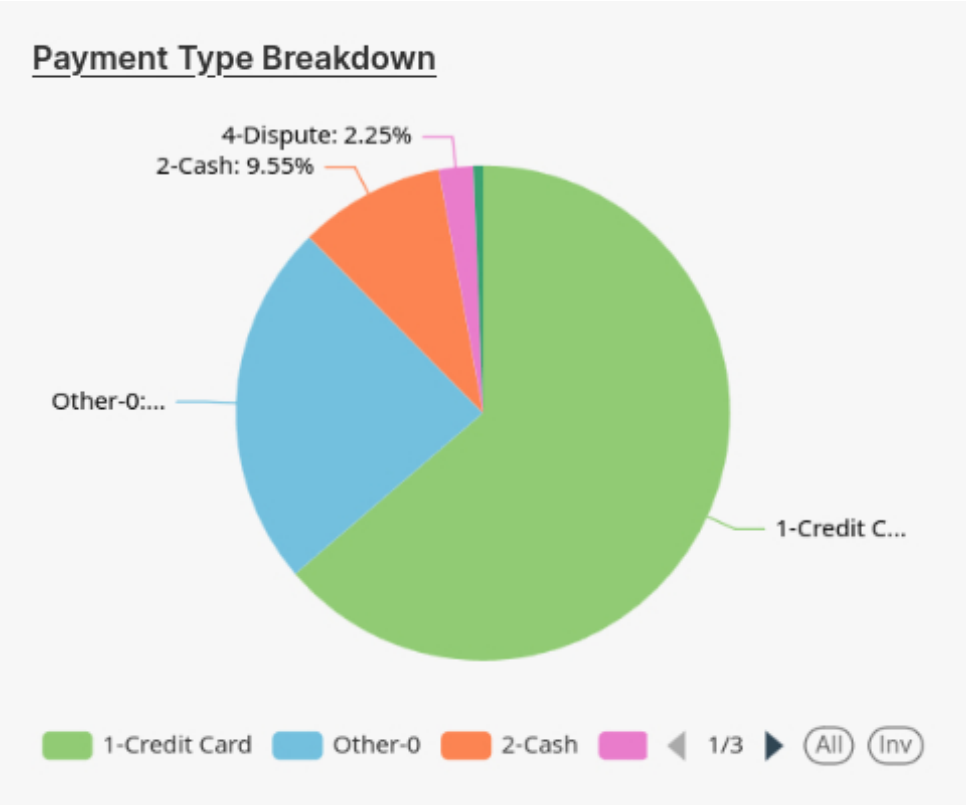

Figure 7: Trip volume and revenue by payment type (Q6).

Credit card payments (type 1) account for 31 million trips (63.7%) and $939 million in revenue. The average tip for credit card trips is $4.34, versus $0.00 for cash trips (type 2) — consistent with the fact that cash tips are not captured in the system. Payment type 0 (unknown or missing) corresponds to 11.6 million trips and overlaps almost perfectly with the NULL passenger count population, reinforcing the hypothesis that this is a vendor-specific data gap.

6.6 Hourly Demand (Q7)

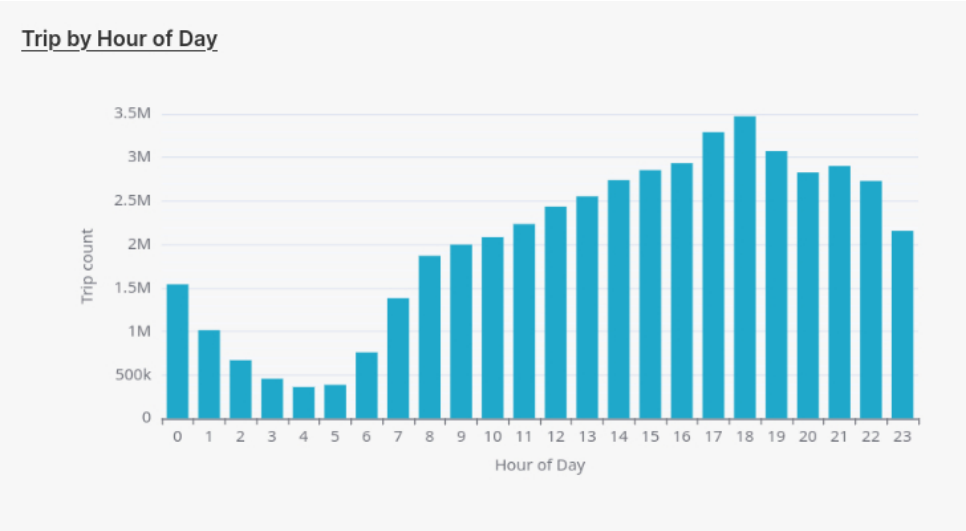

Figure 8: Total number of trips by hour of day (Q7).

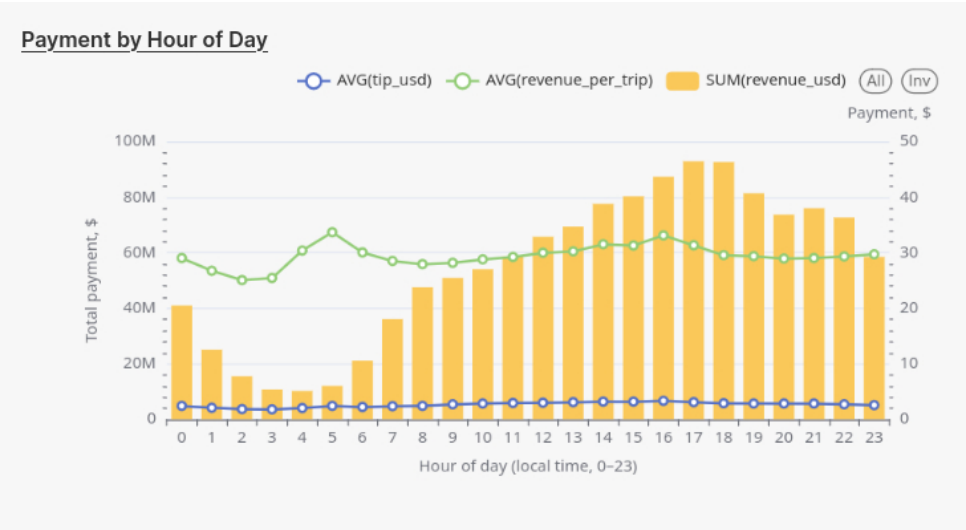

Figure 9: Average revenue per trip and average tip by hour of day (Q7).

Trip demand follows a clear daily rhythm. Volume is at its lowest between 3 and 5 AM, then climbs steadily through the morning commute and peaks during the evening commute window from 17:00 to 19:00, where hourly trip counts exceed 3 million. Interestingly, the highest average revenue per trip occurs in the 4–6 AM slot ($28–31), when the overall volume is low — these early-morning trips are disproportionately airport runs or long-haul dispatches to the outer boroughs, which naturally carry higher fares. Tips follow a similar pattern to revenue, peaking mid-afternoon and declining after midnight.

6.7 Trip Types and Airport Premiums (Q8)

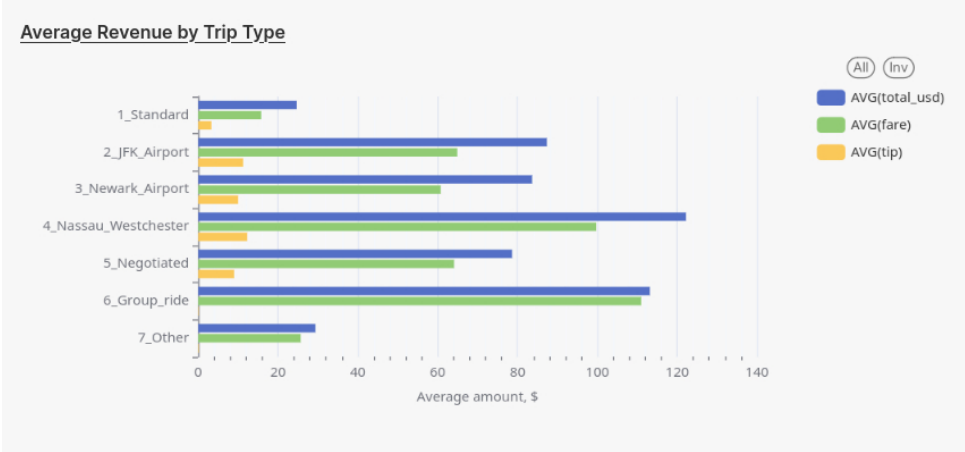

Figure 10: Average total charge and tip by rate code (Q8).

The rate code breakdown reveals a large fare premium for non-standard trips. Nassau/Westchester trips (rate code 4) yield the highest average total charge of $122.20 and an average tip of $12.28, reflecting their long distance (mean 26.34 miles). JFK Airport trips average $87.33 total with an $11.25 tip, and Newark Airport trips average $83.65. Standard metered trips, which make up the vast majority of the dataset (34.3 million records), average $24.68 in total. This confirms that airport and long-distance rate codes are the most lucrative segments for operators on a per-trip basis.

6.8 Revenue per Mile (Q9)

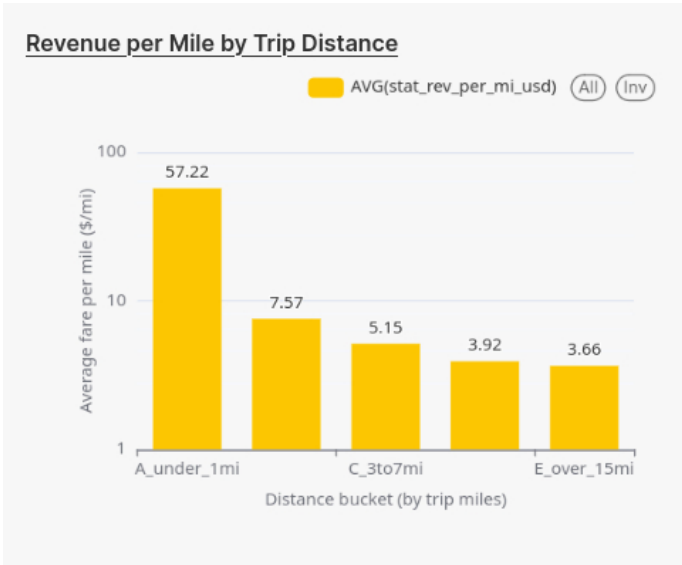

Figure 11: Average fare revenue per mile by distance bucket (Q9).

The revenue-per-mile metric exposes a striking pricing asymmetry. Trips under one mile generate $57.22 per mile on average — more than fifteen times the $3.66 per

mile earned on trips over 15 miles. This is a direct consequence of the NYC taxi minimum fare structure: short trips pay a flat entry charge that dominates the total fare, while long trips are priced closer to the per-mile rate with diminishing returns as distance grows.

6.9 Weekly Patterns (Q10)

Looking at the day-of-week breakdown, Thursday is the busiest weekday with 7.48 million trips and the highest total weekly revenue at $204.6 million. Saturday has the most trips in absolute terms (7.54 million), but its average revenue per trip is the lowest of any day at $25.01, which suggests that weekend rides tend to be shorter social or leisure trips within the city rather than business commutes or airport transfers. Monday, by contrast, shows the highest average revenue per trip at $28.02, consistent with business travel and airport runs at the start of the work week.

6.10 Top Pickup Locations (Q11)

Zone 237 (Upper East Side North) and zone 161 (Midtown Center) are the two highest-volume pickup locations, with over 2 million pickups each and moderate average fares of around $12–16. Zone 132 (JFK International Airport) is notable for its combination of high volume (2.06 million pickups, third overall) and the highest average fare in the top-10 at $56.04 — a direct reflection of the flat-rate JFK fare structure. The top-10 zones are concentrated in Midtown and the Upper East Side, as would be expected for a Manhattan-centric taxi network.

7 Discussion

8 Conclusion

Stage 2 successfully extended the project pipeline into the Apache Hive data warehouse layer. Starting from the Avro files deposited in HDFS by Sqoop in Stage 1, external Hive tables were created, including a monthly-partitioned and vendor-bucketed optimised table that allows efficient querying of the full 48.7-million-record 2025 NYC Yellow Taxi dataset. Eleven EDA queries were formulated. Eight queries were visualised in Apache Superset. The entire pipeline is automated through `scripts/stage2.sh` and produces reproducible outputs in the `output/` folder.

9 Stage 3: ML Modeling

Stage III performs predictive data analysis with Spark MLlib. The objective is to predict the taxi trip tip amount, so the task is formulated as a regression problem. The model is trained on credit-card trips only, because tips paid by cash are not reliably recorded in the dataset. The target variable is `tip_amount`, which is renamed to `label` before model training.

The Stage III pipeline reads the optimized Hive table `yellow_taxi_trips_part_buck`. After feature engineering, the data is split chronologically: approximately the earliest 70% of trips are used for training and the latest 30% are used for testing. This is more realistic than a random split because the model is evaluated on trips that happen later in time.

9.1 Feature Extraction and Data Preprocessing

The preprocessing step first aligns Hive column names with the TLC Parquet naming convention. In Stage II, pickup and drop-off timestamps are stored as epoch milliseconds for AVRO compatibility, so Stage III converts them back to timestamps before extracting time-based features.

Several filters are applied to remove invalid or extreme records. The pipeline keeps only credit-card trips, positive fare amounts, positive trip distances, nonnegative tips, fares below 500 dollars, trip distances below 200 miles, passenger counts from 1 to 6, and trip durations between 1 and 180 minutes. These filters reduce noise from invalid records and extreme outliers before model training.

The selected predictive features include vendor identifier, passenger count, trip distance, rate code, pickup and drop-off location identifiers, fare amount, and the derived pre-tip amount. Time-related features are extracted from the pickup timestamp using cyclical encodings for month, hour, and day of week. Additional binary indicators represent weekend trips, rush-hour trips, night trips, and airport-rate trips. The pre-tip amount is calculated as the sum of fare, taxes, tolls, surcharges, and fees before gratuity.

Categorical variables such as vendor, rate code, pickup location, and drop-off location are processed with `StringIndexer` and `OneHotEncoder`. Numerical variables are imputed using the median strategy. All features are assembled into a Spark ML feature vector with `VectorAssembler`. For the linear regression model, the assembled feature vector is also standardized with `StandardScaler`; for tree-based models, scaling is not required.

9.2 Training and Fine-Tuning

Three Spark MLlib regression models were trained on the YARN cluster: Linear Regression, Random Forest Regression, and Gradient-Boosted Tree Regression. The final modeling dataset contained 30,052,194 credit-card trips after preprocessing. A chronological split was used, with 21,025,239 rows for training and 9,026,955 rows for testing. Hyperparameter selection was performed with 3-fold cross-validation using RMSE as the selection criterion. To keep the grid search feasible on the full taxi dataset, cross-validation was run on a representative random sample of 4,800 rows from the training split; after the best hyperparameters were selected, each final model was refit on the full training split.

Model	**Selected hyperparameters**	**RMSE**	**R^2**
Linear Regression	`regParam=0.0001`, `elasticNetParam=0.0`, `tol=0.0001`, `maxIter=200`	2.5856	0.6186
Random Forest Regressor	`numTrees=50`, `maxDepth=8`, `minInstancesPerNode=10`, `maxBins=32`	2.5768	0.6212
GBT Regressor	`maxDepth=4`, `stepSize=0.03`, `minInstancesPerNode=20`, `maxIter=100`	2.5819	0.6197

Model	Selected hyperparameters	RMSE	R^2

Table 5: Model tuning and evaluation summary

9.3 Evaluation

The final evaluation was performed on the held-out test set of 9,026,955 later trips. All three models produced similar results, with RMSE close to 2.58 dollars and R^2 close to 0.62. The Random Forest Regressor achieved the best RMSE and R^2, with RMSE = 2.5768, MAE = 1.3411, and R^2 = 0.6212. It also predicted 61.58% of test trips within one dollar of the true tip and 80.72% within two dollars.

Model	RMSE	MAE	R^2	Expl. Var.
Linear Regression	2.5856	1.3751	0.6186	0.6187
Random Forest Regressor	2.5768	1.3411	0.6212	10.2820
GBT Regressor	2.5819	1.3392	0.6197	10.4309

Table 6: Core test-set regression metrics from `output/evaluation.csv`

Model	Mean Err.	Median Abs. Err.	P90 Abs. Err.	Within $1	Within $2
Linear Regression	-0.0417	0.7878	2.9797	0.5765	0.8009
Random Forest Regressor	-0.0531	0.6894	2.9140	0.6158	0.8072
GBT Regressor	-0.0432	0.6750	2.8902	0.6152	0.8069

Table 7: Residual and threshold accuracy metrics from `output/evaluation.csv`

The Gradient-Boosted Tree model had the lowest MAE, 1.3392, and the best median absolute error, 0.6750 dollars, but its RMSE and R^2 were slightly worse than the Random Forest model. This suggests that GBT made slightly smaller typical errors, while Random Forest handled larger errors marginally better. Linear Regression was competitive but weaker than both tree-based models, which is expected because tipping behavior depends on nonlinear interactions between trip distance, fare components, time, and pickup/drop-off zones. Based on RMSE and R^2, the Random Forest Regressor is selected as the best final model.

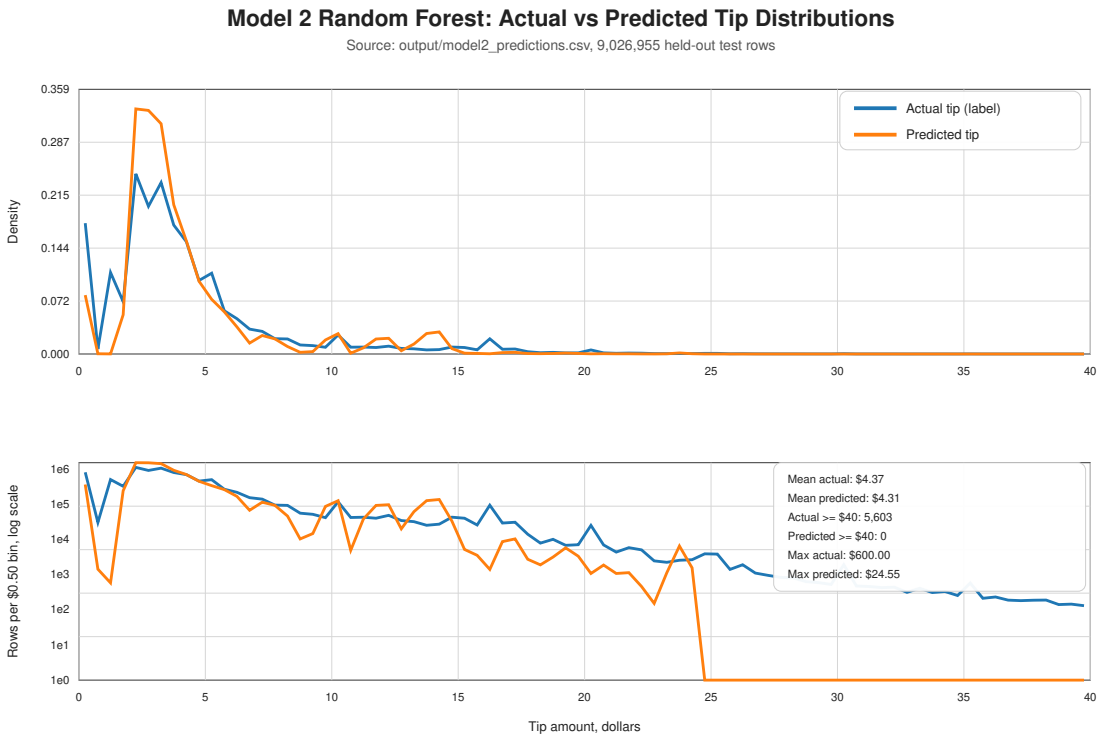

Figure 12: Distribution of actual and predicted tip amounts for the Random Forest model on the held-out test set

9.4 Prediction for a Specific Sample

To illustrate an individual prediction, one trip was selected from the held-out test split and scored with the final Random Forest Regressor. The selected sample is a weekend trip with vendor ID 2, one passenger, trip distance of 2.12 miles, pickup location ID 43, and drop-off location ID 107. The fare amount was 17.00 dollars, and the engineered pre-tip amount, including taxes, fees, tolls, and surcharges before gratuity, was 21.00 dollars. The trip was not during rush hour, was not a night trip, and did not use an airport rate.

Field	**Value**
Trip distance	2.12 miles
Fare amount	$17.00
Pre-tip amount	$21.00
Passenger count	1
Pickup location ID	43
Drop-off location ID	107
Weekend trip	Yes
Actual tip	$3.70
Predicted tip	$3.84
Absolute error	$0.14

Table 8: Example Random Forest prediction for one held-out test trip

For this trip, the model predicted a tip of 3.84 dollars, while the actual recorded tip was 3.70 dollars. The absolute error is only 0.14 dollars, so this example is well within the one-dollar accuracy band used in the evaluation. The prediction is also reasonable relative to the trip's pre-tip amount: both the true and predicted tips are around 18% of the pre-tip cost, which matches common taxi tipping behavior.

10 Stage 4: Data Presentation

We composed an Apache Superset dashboard called *NYC Yellow Taxi 2025: EDA and Tip Prediction*. The published dashboard is available here.

10.1 Dashboard Overview

The dashboard is organized as a story that starts from data quality and descriptive analytics, then moves to trip and revenue patterns, and finishes with predictive modeling results. The first part explains the reliability of the dataset and highlights business patterns in taxi demand, payment behavior, and revenue composition. The final part presents model evaluation, prediction quality, feature importance, and selected hyperparameters from cross-validation.

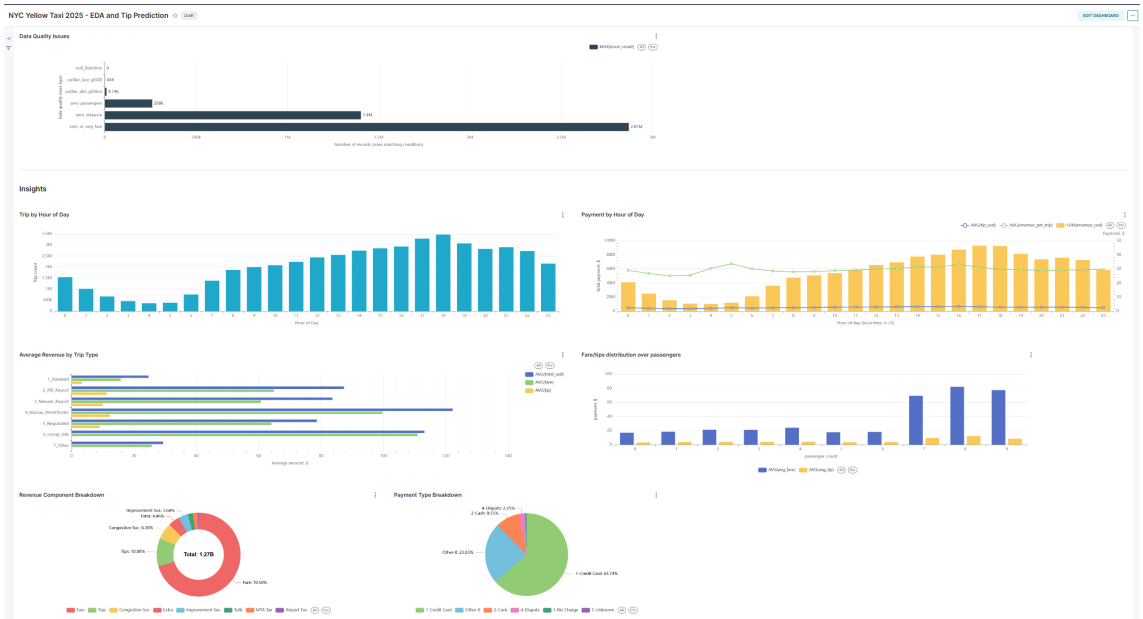

Figure 13: Apache Superset EDA section

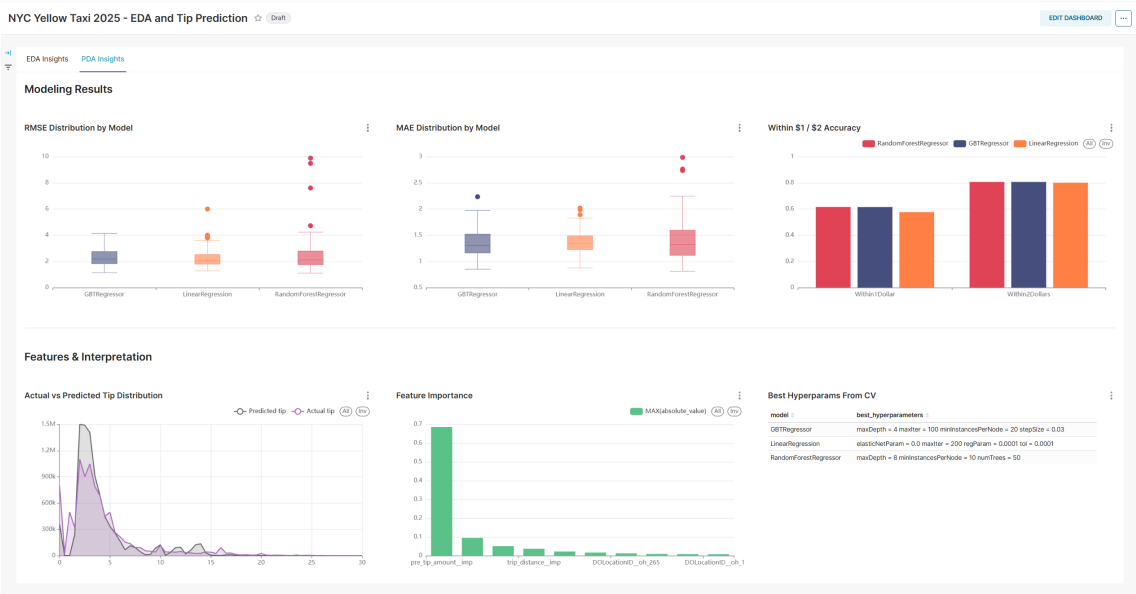

Figure 14: Apache Superset PDA section

10.2 Chart Descriptions

Chart	Purpose	Finding
Data Quality Issues	Bar plot of problematic records, including zero passengers, zero distance, non-positive fares, missing datetimes, and large outliers.	The chart identifies the main data quality risks before analysis and confirms why filtering and cleaning rules are required before modeling.
Trips by Hour of Day	Bar plot showing average tip amount for each of the 24 pickup hours.	Tipping behavior changes during the day, so pickup hour is a useful temporal signal for both EDA and predictive modeling.
Payment by Hour of Day	Combined bar and line chart showing payment totals, average payment amount, and average tip amount by hour.	The chart connects demand periods with revenue and tips, helping identify hours with stronger business value.
Average Revenue by Trip Type	Bar plot comparing average revenue for standard, airport, group, negotiated, and other trip types.	Airport and non-standard rate trips tend to have different revenue profiles from ordinary trips, supporting rate type as an important explanatory variable.

Chart	Purpose	Finding
Fare/tips Distribution over Passengers	Bar plot comparing average fare and average tip by passenger count.	The relationship between passenger count, fare, and tip is not purely linear, so passenger count provides useful context but is not the dominant driver of tips.
Revenue Component Breakdown	Pie chart splitting total revenue into fare, tips, congestion surcharge, airport fee, taxes, tolls, and other components.	Fare is the largest revenue component, while tips and surcharges form meaningful secondary components of total trip income.
Payment Type Breakdown	Pie chart showing the fraction of payments by credit card, cash, dispute, and other payment types.	Credit card trips dominate the modeled subset and are especially relevant because tip amounts are reliably recorded for card payments.
RMSE Distribution by Model	Box plot showing RMSE values across prediction sample buckets for each model.	Tree-based models show competitive error distributions, while the spread indicates how stable each model is across different sampled parts of the test data.
MAE Distribution by Model	Box plot showing MAE values across prediction sample buckets for each model.	MAE highlights typical absolute tip error and supports comparing models in dollar terms that are easy to interpret.
Within $1 / $2 Accuracy	Bar plot showing the share of predictions within one and two dollars of the true tip for each model.	Most predictions fall within the two-dollar band, making the models practically useful for approximate tip estimation.
Actual vs Predicted Tip Distribution	Overlapped area chart comparing actual and predicted tip distributions for the Random Forest model.	The predicted distribution follows the main shape of the actual tip distribution, although rare high-tip trips remain harder to capture.
Feature Importance	Bar plot of the top 10 GBTRegressor feature importances.	Fare-related features, especially pre-tip amount and fare amount, are the strongest predictors of tip amount.
Best Hyperparameters from CV	Table of selected cross-validation hyperparameters for Linear Regression, Random Forest, and GBT.	The table documents the tuned model configurations and makes the modeling stage reproducible.

Chart	Purpose	Finding

Table 9: Dashboard chart descriptions

10.3 Findings

The dashboard and modeling results show that taxi profitability is driven by a combination of trip context, payment behavior, fare structure, and time. Data quality checks revealed several problematic record types, including zero passengers, zero distance, non-positive fares, missing timestamps, and extreme outliers. These issues justify the filtering rules used before model training and help explain why raw trip data should not be used directly for predictive analytics.

Exploratory analysis showed that trips differ strongly by hour of day, trip type, and passenger count. Hourly charts indicate that demand, payment totals, and average tips vary across the day, which supports the use of pickup-hour features in the model.

The predictive analysis confirms that tip amount can be estimated with useful accuracy from trip and fare characteristics. All three Spark ML models achieved similar performance on the held-out chronological test set, with RMSE around 2.58 dollars and R^2 around 0.62. Feature-importance charts showed that fare-related variables, especially pre-tip amount and fare amount, are the strongest predictors, followed by trip distance, rate code, and location-related features.

11 Conclusion

This project built an end-to-end big data pipeline for analyzing NYC Yellow Taxi trips from 2025 and predicting passenger tip amount. The dataset contained trip records with timestamps, locations, passenger counts, payment types, fare components, and tip amounts. In Stage 1, the raw Parquet data was loaded into PostgreSQL and imported to HDFS with Sqoop as Snappy-compressed Avro files. In Stage 2, Hive external tables were created over the HDFS data, including a partitioned and bucketed analytical table used for EDA and downstream modeling. In Stage 3, Spark MLlib models were trained on the optimized Hive table to predict `tip_amount` for credit-card trips. In Stage 4, the EDA outputs and model artifacts were presented through an Apache Superset dashboard.

The analytical part of the project identified meaningful business patterns in data quality, hourly demand, payment behavior, trip type, passenger count, and revenue composition. These insights show that trip profitability is not explained by one variable alone: fare amount, time of day, rate code, payment method, surcharges, and trip context all contribute to the observed outcomes.

The modeling part demonstrated that large-scale tip prediction is feasible with distributed machine learning. Linear Regression, Random Forest, and Gradient-Boosted Trees were trained and evaluated on a chronological train/test split. The tree-based models performed best overall, with Random Forest selected as the final model based on RMSE and R^2. Its performance is accurate enough for approximate business estimation, especially because more than 80% of predictions were within two dollars of the actual tip.

The final Superset dashboard combines descriptive analytics and predictive results in one presentation layer. It allows stakeholders to inspect dataset quality, explore revenue and tipping patterns, compare model performance, examine prediction distributions, and understand the most important features used by the models.

12 Testing and Quality Assurance

The project includes a multi-layered automated testing system designed to verify correctness without requiring a live Hadoop cluster in CI.

12.1 Static Tests (GitHub Actions CI)

A GitHub Actions workflow (`.github/workflows/ci.yml`) runs automatically on every push and pull request. It performs the following checks in an Ubuntu runner without any cluster dependencies:
Shell script linting: `shellcheck` validates all `.sh` files for common Bash pitfalls (quoting, variable expansion, portability).
Python linting: `ruff` checks all Python files for formatting and import issues; `pylint` evaluates code quality with a configurable score threshold.
Bash syntax check: `bash -n` verifies that every shell script parses without errors.
Python compile check: `py_compile` ensures all Python modules compile successfully.
Pytest static test suite: 254 unit tests verify the project structure and content statically:

- `test_repo_structure.py` — repository layout, `.gitignore` coverage, output artifacts.
- `test_security.py` — detection of hardcoded passwords or secrets in source files.
- `test_shell_scripts.py` — script existence, shebangs, pipeline execution order in `main.sh`.
- `test_stage_scripts.py` — rubric-specific checks per stage (Sqoop options, Hive configuration, Spark-on-YARN arguments).
- `test_sql_hql_static.py` — SQL/HQL content validation: schema definitions, partitioning, bucketing, EDA query count, aggregation patterns.
- `test_pyspark_ml_static.py` — PySpark ML code validation: pipeline components, three model types, cross-validation setup, hyperparameter counts, model/prediction saving conventions.
- `test_static_validation.py` — integration tests for the core `static_validate.py` analysis module.

All static tests read source files and check patterns, imports, and structural properties. They do not execute any pipeline code or require cluster access.

12.2 Cluster Verification Scripts

Two Bash scripts are provided for manual execution on the Hadoop cluster after running the pipeline:

- `tests/cluster_verify.sh` performs a comprehensive rubric compliance check: PostgreSQL table existence and schema (via `psycopg2`), HDFS directory structure for Sqoop and Hive warehouses, partition directories, local model and prediction artifacts, evaluation CSV content, and pylint quality score. It reports PASS/WARN/FAIL for each check.
- `tests/cluster_smoke.sh` runs the actual pipeline stages (`stage1.sh`, `stage2.sh`, `stage3.sh`) and verifies expected outputs, intended for full end-to-end validation.

Neither cluster script is executed by GitHub Actions CI.

12.3 Testing Challenges

The main challenge was designing tests that provide meaningful coverage without access to the Hadoop cluster, HDFS, Hive, or PostgreSQL in the CI environment. The solution was a two-tier architecture: static tests that validate code structure and patterns run in CI, while runtime verification scripts run separately on the cluster. Additional issues included beeline timeouts on large Hive tables (resolved by using HDFS metadata checks instead of live queries).

13 Reflections on Own Work

The project required coordination across several dependent pipeline stages. The team divided responsibilities by stage, with one person mainly responsible for each major part of the pipeline, while Nikita contributed to testing Stages I–III. This division made ownership clear, but it also introduced dependencies between teammates. For example, Stage III could not be fully executed and validated before Stages I and II were completed, because the machine learning pipeline depends on the prepared Hive table and exported data artifacts. As a result, Stage III and testing had less effective time available than the earlier stages.

Team collaboration was also affected by technical access constraints. Some team members needed VPN access to work with the cluster environment, and resolving these access issues consumed additional time. Overall, the project showed that in a multi-stage big data pipeline, technical readiness and early integration are as important as the implementation of individual scripts.

13.1 Challenges and Difficulties

The main organizational challenge was the sequential dependency between stages. Data ingestion, Hive preparation, machine learning, and testing could not be completed fully independently, because later stages depended on outputs generated by earlier stages. This made delays in one part of the pipeline affect the time available for downstream work.

The main technical challenge was resource management on the cluster. Initially, the stages were implemented without strict memory limitations, but later a 20 GB quota was introduced. This caused out-of-memory errors during Spark execution and model tuning. To continue the work, the team had to request a larger quota and adjust execution settings for the cluster environment.

Pipeline performance was another difficulty. Loading, transforming, querying, and modeling tens of millions of taxi trip records takes a long time, especially when running cross-validation for multiple machine learning models. Because every model tuning experiment is computationally expensive, the final hyperparameter search may not be globally optimal, but it represents a practical compromise between model quality, available time, and cluster resource limits.

13.2 Recommendations

For future work, the team should integrate the stages earlier and maintain small validation samples for every stage. This would allow downstream work, especially machine learning and testing, to start before the full dataset is processed. A lightweight sample pipeline would also make debugging faster and reduce dependence on long cluster jobs.

The machine learning stage could be improved by adding more structured experiment tracking and by running a broader hyperparameter search when more resources are available. Additional features, such as weather, traffic conditions, public events, or taxi zone metadata, could also improve tip prediction quality and make the dashboard more useful for business interpretation.

The pipeline now includes a comprehensive automated testing layer (254 static Pytest tests in CI and a cluster verification script), which significantly reduces the risk of discovering integration problems late. Future work could extend this with automated regression testing for model quality metrics and more granular per-stage integration tests. Finally, the presentation layer should be completed with a stable dashboard workflow so that analytical and predictive results can be updated directly from pipeline outputs.

13.3 Team Contributions

Task	Description	A.N.	V.A.	U.C.	N.S.	Deliverables	Hours

Stage 1	PostgreSQL schema creation, Parquet loading, Sqoop import to HDFS, AVRO schema extraction, and ingestion validation	0%	100%	0%	0%	`sql/create_tables.sql`
						`scripts/build_projectdb.py`
						`scripts/stage1.sh`
						HDFS AVRO warehouse
						`output/yellow_taxi_trips.avsc`
							4

Stage 2	Hive database creation, external AVRO table setup, partitioned and bucketed table preparation, 11 EDA query design, CSV export, and Superset chart creation and export (8 visualizations)	0%	0%	100%	0%	`sql/db.hql`
						`sql/eda.hql`
						`scripts/stage2.sh`
						`output/q1.csv–output/q11.csv`
						`output/hive_results.txt`
						`output/eda_results.txt`
						`output/*.jpg` (8 charts)
							18

Stage 3	Spark ML regression: feature engineering, chronological train-test split, cross-validation, model training, evaluation, and output export	100%	0%	0%	0%	`scripts/stage3.py`
						`scripts/stage3.sh`
						trained models
						prediction CSVs
						evaluation metrics
							20

Task	Description	A.N.	V.A.	U.C.	N.S.	Deliverables	Hours
Stage 4	Dashboard preparation and presentation planning using EDA outputs, model metrics, predictions, and feature signals	50%	0%	50%	0%	Dashboard charts EDA visualizations model performance views prediction results `scripts/stage4.sh` placeholder	4

Testing	Static test suite (254 Pytest tests), CI workflow, cluster verification script, cluster smoke script, security and linting checks	0%	0%	0%	100%	`.github/workflows/ci.yml`
						`tests/test_*` (7 modules)
						`tests/static_validate.py`
						`tests/cluster_verify.sh`
						`tests/cluster_smoke.sh`
						`Makefile`
							12

Report and presentation	Report writing, structure preparation, result interpretation, and presentation materials	30%	30%	20%	20%	Final report presentation slides	5

Table 10: Team contribution table